

Support élève (Thème 1)



Activité 1 : Programmation fonctionnelle

Question 1. On souhaite sélectionner uniquement les élèves ayant obtenu au moins 10. Écrire une fonction `est_admis` qui reçoit un couple (`nom`, `note`) et renvoie :

- `True` si la note est supérieure ou égale à 10 ;
- `False` sinon.

Par exemple :

```
1      est_admis(("Alice", 15)) # doit renvoyer True
```

Utiliser ensuite la fonction `filter` pour sélectionner les élèves admis.

Question 2. On souhaite maintenant récupérer uniquement les noms des élèves admis. Écrire une fonction `recuperer_nom` qui reçoit un couple (`nom`, `note`) et renvoie le nom de l'élève.

Par exemple :

```
1      recuperer_nom(("Alice", 15)) # doit renvoyer "Alice"
```

Utiliser ensuite `map` pour appliquer cette fonction aux élèves admis.

On souhaite obtenir :

```
1      ['Alice', 'Chloé', 'David', 'Farid', 'Gabriel', 'Hugo', 'Inès']
```

Question 3. On souhaite maintenant récupérer uniquement les notes des élèves admis.

Écrire une fonction `recuperer_note` qui reçoit un couple (`nom`, `note`) et renvoie la note.

Par exemple :

```
1      recuperer_note(("Alice", 15)) # doit renvoyer 15
```

Utiliser `map` pour obtenir la liste des notes des élèves admis.

Résultat attendu :

```
1      [15, 12, 17, 14, 19, 10, 13]
```

Question 4. On souhaite maintenant calculer la somme des notes des élèves admis.

Importer la fonction `reduce` :

```
1 from functools import reduce
```

Écrire une fonction `additionner` qui reçoit deux nombres `x` et `y` et renvoie leur somme.

Utiliser `reduce` pour calculer la somme des notes obtenues à la question précédente.

Résultat attendu :

```
1 100
```

Question 5. On souhaite maintenant calculer la moyenne des élèves admis.

Utiliser :

- `filter` pour sélectionner les élèves admis ;
- `map` pour récupérer leurs notes ;
- `reduce` pour calculer leur somme ;
- `len` pour connaître leur nombre.

Le programme doit afficher :

```
1 Nombre d'admis : 7
2 Somme des notes : 100
3 Moyenne : 14.29
```



Activité débranchée 1 : Introduction

```

1  class Personne:
2      def __init__(self, lastName, firstName):
3          self.nom = lastName
4          self.prenom = firstName
5
6  eleve1 = Personne('NOM', 'Prenom') #à remplacer par les vôtres
7  eleve2 = Personne('NOM', 'Prenom') #à remplacer par les vôtres
8  eleve3 = Personne('NOM', 'Prenom') #à remplacer par les vôtres
9
10 class Chaise:
11     def __init__(self):
12         self.occupe = None
13         self.direction = 'N' #Nord
14
15     def occuper(self, eleve : Personne) -> None :
16         self.occupe = eleve
17
18     def liberer(self) -> None :
19         self.occupe = None
20
21     def est_occupee(self) -> bool :
22         return self.occupe is not None
23
24     def orienter(self, point_cardinal:str) -> None:
25         """oriente la chaise suivant les directions N (nord), S (sud), E (est), O (ouest)"""
26         self.direction = point_cardinal
27
28     def get_direction(self) -> str:
29         return self.direction
30

```

1. Numéroté les chaises puis matérialiser avec les chaises et les élèves de la classe l'application des instructions suivantes :

```

1      ch1 = Chaise()
2      ch2 = Chaise()
3      ch3 = Chaise()
4      ch2.occuper(eleve1)
5      ch2.orienter('E')

```

Faire vérifier par le professeur pour continuer.

```

1      ch3.occuper(eleve2)
2      ch3.orienter('S')
3      ch2.liberer()
4      ch2.orienter('O')

```

Faire vérifier par le professeur pour continuer.

2. A ce stade, que donnent les instructions suivantes ?

- a) ch1.est_occupee() ? et ch1.get_direction() ? et ch1.direction ?
- b) ch2.est_occupee() ?
- c) ch2.get_direction() ?
- d) ch3.est_occupee() ?
- e) ch3.direction ?

3. Pourquoi utilise-t-on des parenthèses pour `ch1.get_direction()` alors qu'on n'en utilise pas pour `ch1.direction` ?

```

1 class Pile_de_chaises :
2     def __init__(self):
3         self.pile = []
4     def empiler(self, chair:Chaise) -> None:
5         if chair.occupe == None :
6             if len(self.pile) > 0 and chair.direction == self.pile[-1].direction :
7                 self.pile.append(chair)
8             elif len(self.pile)==0 :
9                 self.pile.append(chair)
10
11     def diminuer_la_pile(self) -> None:
12         if len(self.pile)>1:
13             self.pile.pop()
14
15     def nb_chaises_empilees(self) -> int :
16         return len(self.pile)

```

4. Matérialiser avec les chaises et les élèves de la classe l'application des instructions suivantes :

<pre> 1 #Scénario 1 2 ch4=Chaise() 3 ch4.orienter('N') 4 5 ch5=Chaise() 6 ch5.orienter('O') 7 8 ch6=Chaise() 9 ch6.orienter('N') 10 11 ch7=Chaise() 12 ch7.occuper(eleve3) 13 14 ma_pile = Pile_de_chaises() 15 16 ma_pile.empiler(ch4) 17 ma_pile.empiler(ch5) 18 ma_pile.empiler(ch6) 19 ma_pile.empiler(ch7) 20 21 print(ma_pile.nb_chaises_empilees()) 22 23 24 "Faire vérifier par le professeur pour continuer." </pre>	<pre> #Scénario 2 (Suite du scénario 1) ch7.liberer() ch5.orienter('S') ch8=Chaise() ch8.orienter('S') ch8.occuper(eleve1) ma_pile.diminuer_la_pile() ma_pile.empiler(ch5) ma_pile.empiler(ch7) ma_pile.empiler(ch8) print(ma_pile.nb_chaises_empilees()) "Faire vérifier par le professeur." </pre>
---	---



Exercice 1 : C'est la classe

Soit le programme suivant :

```
1 class Ennemi:
2     def __init__(self):
3         self.point_de_vie = 20
4         self.difficulte_ia = "Facile"
5
6     def est_vaincu(self):
7         return self.point_de_vie <= 0
8
9     def degats_subis(self, degat):
10        self.point_de_vie -= degat
11
12    def mise_a_jour_ia(self):
13        if self.point_de_vie < 13:
14            self.difficulte_ia = "Moyenne"
15        elif self.point_de_vie < 5:
16            self.difficulte_ia = "Difficile"
17
18    mechant = Ennemi()
19
20    degat = 1
21    while not mechant.est_vaincu():
22        mechant.degats_subis(degat)
23        mechant.mise_a_jour_ia()
24        degat += 1
```

Question 1. Identifier le type de la variable `mechant`.

Question 2. Lister les attributs et méthodes de la classe `Ennemi`.

Question 3. Quelles sont les valeurs des attributs de l'objet `mechant` à sa création ?

Question 4. Noter à chaque tour de la boucle, les valeurs des attributs de `mechant`.



Activité 2 : Classe-ment

Question 1. Implémenter la classe `Ennemi` donnée précédemment. Puis lui ajouter un nouvel attribut nommé `arme`. Cet attribut correspond à un tuple (nom de l'arme, dégât de l'arme).

Question 2. Ajouter une méthode `prendre_arme` qui met à jour l'arme de l'Ennemi. Cette méthode prend en paramètres un nom d'arme et des dégâts.

Question 3. Ajouter ensuite une méthode permettant à la classe `Ennemi` de tirer nommée `faire_degat` et qui renvoie les dégâts de l'arme.

Question 4. Créer deux instances de la classe `Ennemi`. Puis effectuer un tirage aléatoire de valeurs, si la valeur est paire alors c'est le premier ennemi qui tire, réduisant la vie du second. Si la valeur est impaire c'est l'inverse. Réaliser des tirages aléatoires de valeurs jusqu'à la mort d'un des Ennemis et afficher le nombre de points de vie restant du vainqueur.



Activité 3 : Bot-aille

On récupère la classe `JeuDeCartes` suivante, les attributs ont été complétés, mais les méthodes non.

```
1 class JeuDeCartes:
2
3     def __init__(self):
4         """Construit un jeu de 52 cartes."""
5         self.cartes = []
6
7         self.valeurs = []
```

```

8         2, 3, 4, 5, 6, 7, 8, 9, 10,
9         "valet", "dame", "roi", "as"
10    ]
11
12    self.couleurs = [
13        "Pique", "Trèfle", "Carreau", "Coeur"
14    ]
15
16    # Création des 52 cartes
17    # À compléter (Question 1)
18
19    def nomCarte(self, c):
20        """Renvoie le nom d'une carte."""
21        # À compléter (Question 2)
22
23    def battre(self):
24        """Mélange les cartes."""
25        # À compléter (Question 3)
26
27    def tirer(self):
28        """Retire et renvoie une carte."""
29        # À compléter (Question 4)
30
31    # À compléter (Questions 5 et 6 )

```

On souhaite simuler le jeu de la bataille, utilisant 52 cartes. On sépare les cartes par ce qu'on appelle les couleurs (Trèfle, Carreau, Pique, Cœur). Chaque carte possède également une valeur (2, 3, 4, 5, 6, 7, 8, 9, 10, Valet, Dame, Roi, As).

Question 1. Compléter la méthode `init` de la classe `JeuDeCartes`, de façon à ce que l'attribut `cartes` contienne toutes les cartes du jeu. Autrement dit, que la liste contienne tous les tuples (valeur, couleur) possibles afin de représenter les cartes existantes dans un jeu de 52 cartes.

Question 2. Désormais, compléter la méthode `nomCarte` permettant d'afficher le nom d'une carte. (Ex : As de Pique)

Question 3. Compléter la méthode `battre` permettant de mélanger les cartes. Pour cela, utiliser la méthode `shuffle` de la bibliothèque `random`.

Question 4. Compléter la méthode `tirer` permettant de tirer la carte à l'indice 0 et de la supprimer de la liste de l'attribut `cartes`. La méthode retourne le tuple correspondant à la carte. Si toutes les cartes sont tirées, il faudra retourner `None`.

Question 5. Vérifier la méthode `tirer` en retirant toutes les cartes de la liste une par une avec une boucle `for`.

Question 6. Simuler un jeu de bataille entre deux bots (= joueur robot). Le joueur qui remporte le pli gagne un point. Vous avez le droit d'ajouter des méthodes si nécessaire.